



Logic Map — FastConnect

Assignment #03: REST API Integration & SQLite Persistence

Course: Mobile Application Development | App: FastConnect (FAST University Campus App) | Date: April 25, 2026

F1 — REST API Integration

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F1	NewsApiService.kt	getTopHeadlines()	Retrofit suspend function that sends a GET request to <code>newsapi.org/v2/top-headlines</code> with country and category parameters. Returns <code>NewsResponse</code> containing a list of <code>NewsArticle</code> objects parsed from JSON via Gson.
F1	NewsApiService.kt	companion object (RetrofitClient)	Singleton Retrofit instance with OkHttp logging interceptor. Uses lazy initialization so only one instance is created throughout the app lifecycle. Base URL set to <code>https://newsapi.org/</code> .
F1	NewsFragment.kt	fetchNews()	Uses <code>lifecycleScope.launch</code> with <code>withContext(Dispatchers.IO)</code> to call the API on a background thread. Filters removed articles and updates the RecyclerView adapter. Handles loading/error/empty states.
F1	NewsAdapter.kt	onBindViewHolder()	Binds API data (title, description, source name, formatted date) to <code>item_news.xml</code> card views. Uses ViewHolder pattern for efficient scrolling.
F1	NewsArticle.kt	(data class)	Model mapping API JSON fields: title, description, url, urlToImage, publishedAt, and nested NewsSource object. Implements Serializable.
F1	NewsResponse.kt	(data class)	Wrapper for the full API response body (status, totalResults, articles list). Used by Retrofit's Gson converter.

F2 — SQLite Schema Design

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F2	FastConnectDbHelper.kt	onCreate(db)	Creates two tables using CREATE TABLE SQL statements: • folders (parent): _id PK AUTOINCREMENT, name TEXT UNIQUE, created_at TEXT • bookmarks (child): _id PK AUTOINCREMENT, title, url, note, folder_id (FK → folders._id ON DELETE CASCADE), created_at Enables PRAGMA foreign_keys = ON. Inserts 4 default folders.
F2	FastConnectDbHelper.kt	onOpen(db)	Ensures foreign key enforcement is enabled every time the database connection opens by executing PRAGMA foreign_keys = ON.
F2	Bookmark.kt	(data class)	Model for child table: id, title, url, note, folderId (FK reference), folderName (populated via JOIN), createdAt.
F2	BookmarkFolder.kt	(data class)	Model for parent table: id, name, createdAt. Bookmarks reference folders through folderId FK.



Database Schema (ER Relationship)

Table: folders (Parent)

Column	Type	Constraint
_id	INTEGER	PRIMARY KEY AUTOINCREMENT
name	TEXT	NOT NULL UNIQUE
created_at	TEXT	NOT NULL

Table: bookmarks (Child)

Column	Type	Constraint
_id	INTEGER	PRIMARY KEY AUTOINCREMENT
title	TEXT	NOT NULL
url	TEXT	NOT NULL
note	TEXT	DEFAULT ''
folder_id	INTEGER	FK → folders(_id) ON DELETE CASCADE
created_at	TEXT	NOT NULL

Relationship: One folder → many bookmarks (1:N). Deleting a folder cascades to delete all its bookmarks.

F3 — Persistent CRUD Operations

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F3-C	FastConnectDbHelper.kt	insertBookmark()	CREATE: Builds a ContentValues object with title, url, note, folderId, and timestamp, then calls db.insert() on the bookmarks table. Returns the row ID.
F3-C	FastConnectDbHelper.kt	insertFolder()	CREATE: Inserts a new folder with name and timestamp into the folders table using db.insert().
F3-C	AddEditBookmarkFragment.kt	saveBookmark()	Validates form input (title, URL required), gets selected folder from spinner, then calls dbHelper.insertBookmark() inside withContext(Dispatchers.IO) for background execution.
F3-R	FastConnectDbHelper.kt	getAllBookmarks()	READ: Executes a SQL JOIN query: SELECT b.*, f.name FROM bookmarks b LEFT JOIN folders f ON b.folder_id = f._id ORDER BY created_at DESC. Iterates cursor and builds Bookmark list.
F3-R	BookmarksFragment.kt	loadBookmarks()	Calls the appropriate DB read method on Dispatchers.IO, then updates the RecyclerView adapter and empty state visibility on the main thread.
F3-U	FastConnectDbHelper.kt	updateBookmark()	UPDATE: Builds ContentValues with new title, url, note, folderId and calls db.update() with WHERE _id = ?.
F3-U	AddEditBookmarkFragment.kt	saveBookmark() [edit mode]	When editingBookmark is non-null, calls dbHelper.updateBookmark() instead of insert. Bookmark passed via Bundle (Serializable).
F3-D	FastConnectDbHelper.kt	deleteBookmark()	DELETE: Calls db.delete() with WHERE _id = ? to remove a bookmark row.
F3-D	BookmarksFragment.kt	deleteBookmark()	Calls dbHelper.deleteBookmark() on Dispatchers.IO, then reloads the list. Called from confirmation dialog.
F3-D	BookmarksFragment.kt	confirmDelete()	Shows AlertDialog before deleting to prevent accidental data loss. On confirm, calls deleteBookmark().

F4 — Data Integration Strategy

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F4-B	DashboardActivity.kt	setOnItemSelectedListener	Strategy B: API (News tab) and SQLite (Bookmarks tab) operate as two separate, independent functional modules within the same app. Navigation handled via BottomNavigationView with 5 tabs.
F4	NewsFragment.kt	saveArticleAsBookmark()	Bridge Feature: Long-pressing a news article saves it into SQLite bookmarks under the "Saved News" folder, connecting the API module with the database module. ENHANCED

F5 — Dynamic SQL Queries

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
F5-LIKE	FastConnectDbHelper.kt	searchBookmarks(query)	Executes: SELECT ... WHERE title LIKE '%query%' OR url LIKE '%query%' OR note LIKE '%query%'. Uses parameterized queries to prevent SQL injection.
F5-LIKE	BookmarksFragment.kt	searchBookmarks(query)	Called from SearchView's onQueryTextChanged listener. Runs the SQL LIKE query on Dispatchers.IO and updates the RecyclerView.
F5-ORDER	FastConnectDbHelper.kt	getBookmarksSorted(column, asc)	Executes: SELECT ... ORDER BY [column] [ASC DESC]. Validates column name against a whitelist to prevent SQL injection. Supports sorting by title or created_at.
F5-ORDER	BookmarksFragment.kt	setupSortSpinner()	Spinner with options: "Newest First", "Title A-Z", "Title Z-A", "Oldest First". Each selection triggers loadBookmarks() which calls the appropriate ORDER BY query.
F5-WHERE	FastConnectDbHelper.kt	getBookmarksByFolder(folderId)	Executes: SELECT ... WHERE folder_id = ?. Filters bookmarks belonging to a specific folder category.
F5-WHERE	BookmarksFragment.kt	setupFolderSpinner()	Folder filter spinner showing "All Folders" + all folder names. Selecting a folder triggers WHERE-based filtering.

Background Threading (Kotlin Coroutines)

REQ ID	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
Thread	NewsFragment.kt	fetchNews()	Network call wrapped in lifecycleScope.launch { withContext(Dispatchers.IO) { api.getTopHeadlines() } }. Prevents UI freeze during API calls.
Thread	BookmarksFragment.kt	loadBookmarks(), searchBookmarks(), deleteBookmark()	All SQLite read/write operations wrapped in withContext(Dispatchers.IO) to execute on background threads. UI updates happen automatically on main thread after suspend returns.
Thread	AddEditBookmarkFragment.kt	saveBookmark(), loadFolders(), showCreateFolderDialog()	All DB insert/update operations and folder loading use withContext(Dispatchers.IO) within viewLifecycleOwner.lifecycleScope.launch.

Enhanced Usability Features BONUS

FEATURE	CLASS / FILE	FUNCTION / METHOD	IMPLEMENTATION DESCRIPTION
Save News → Bookmarks	<code>NewsFragment.kt</code>	<code>saveArticleAsBookmark()</code>	Long-press on a news article shows AlertDialog to confirm saving. Inserts article into SQLite with "Saved News" folder. Bridges API and DB modules.
Share	<code>NewsFragment.kt</code> / <code>BookmarksFragment.kt</code>	<code>shareArticle()</code> / <code>shareBookmark()</code>	Creates Intent.ACTION_SEND with text/plain MIME type and launches share chooser (WhatsApp, email, etc.).
Folder Management	<code>AddEditBookmarkFragment.kt</code>	<code>showCreateFolderDialog()</code>	AlertDialog with EditText for entering folder name. Inserts into SQLite and refreshes the folder spinner.
Swipe-to-Delete + Undo	<code>BookmarksFragment.kt</code>	<code>setupSwipeToDelete()</code>	ItemTouchHelper with LEFT swipe. Deletes bookmark, shows Snackbar with "UNDO" action that re-inserts the bookmark if tapped.
Pull-to-Refresh	<code>NewsFragment.kt</code>	<code>swipeRefresh.setOnRefreshListener</code>	SwipeRefreshLayout wrapping the news RecyclerView. Pulling down triggers fetchNews() to reload API data.
Empty States	<code>fragment_news.xml</code> / <code>fragment_bookmarks.xml</code>	<code>showEmpty()</code> / <code>updateEmptyState()</code>	Friendly empty-state views with emoji icons shown when no data available. Visibility toggled based on list content.
Confirmation Dialogs	<code>BookmarksFragment.kt</code>	<code>confirmDelete()</code>	AlertDialog before destructive delete operations to prevent accidental data loss.